

LAB8: RD PARSER FOR DECLARATION STATEMENTS, RD PARSER FOR ARRAY DECLARATIONS AND EXPRESSION STATEMENTS

CSE A2

Vivek Singh

Roll No: 50

230905408

1. For given subset of grammar 7.1, design RD parser with appropriate error messages with expected character and row and column number.

Program → main () { declarations assign_stat }
 declarations → data-type identifier-list; declarations | ∈
 data-type → int | char
 identifier-list → id | id, identifier-list
 assign_stat → id=id; | id = num;

Design the recursive descent parser to parse array declarations and expression statements with error reporting.

Subset of grammar 7.1 is as follows:

```

Program → main () { declarations statement-list }
identifier-list → id | id, identifier-list | id[number] , identifier-list | id[number]
statement_list → statement statement_list | ∈
statement → assign-stat;
assign_stat → id = expn
expn → simple-expn eprime
epime → relop simple-expn | ∈
simple-exp → term seprime
seprime → addop term seprime | ∈
term → factor tprime
tprime → mulop factor tprime | ∈
factor → id | num
relop → == | != | <= | >= | > | <
addop → + | -
mulop → * | / | %

statement → assign-stat; | decision_stat | looping-stat
decision-stat → if (expn) {statement_list} dprime
    
```

Q1

```

#include "getToken.c"
#include <stdlib.h>
    
```

```
int cur = 1;
```

```

void invalid()
{
    printf("Invalid, %s\n", tokarr[cur].name);
    printf("r = %d, c = %d\n", tokarr[cur].row, tokarr[cur].col);
    exit(0);
}
    
```

```

void valid()
{
    printf("Parsing done successfully\n");
}
    
```

```

}

void Program();
void declarations();
void data_type();
void identifier_list();
void l1();
void statement_list();
void statement();
void assign_stat();
void decision_stat();
void dprime();
void looping_stat();
void expn();
void eprime();
void simple_expn();
void seprime();
void term();
void tprime();
void factor();
void relop();
void addop();
void mulop();

void Program()
{
    if (strcmp(tokarr[cur].name, "main") == 0)
    {
        cur++;
        if (strcmp(tokarr[cur].name, "(") == 0)
        {
            cur++;
            if (strcmp(tokarr[cur].name, ")") == 0)
            {
                cur++;
                if (strcmp(tokarr[cur].name, "{") == 0)
                {
                    cur++;
                    declarations();
                    statement_list();
                    if (strcmp(tokarr[cur].name, "}") == 0)
                    {
                        cur++;
                        return;
                    }
                    else
                        invalid();
                }
                else
                    invalid();
            }
            else
                invalid();
        }
        else
            invalid();
    }
    else
        invalid();
}
}

```

```

void declarations()
{
    if (strcmp(tokarr[cur].name, "int") == 0 || strcmp(tokarr[cur].name, "char") == 0)
    {
        data_type();
        identifier_list();
        if (strcmp(tokarr[cur].name, ";") == 0)
        {
            cur++;
            declarations();
            return;
        }
        else
            invalid();
    }
    else
    {
        return;
    }
}

```

```

void data_type()
{
    if (strcmp(tokarr[cur].name, "int") == 0)
    {
        cur++;
        return;
    }
    else if (strcmp(tokarr[cur].name, "char") == 0)
    {
        cur++;
        return;
    }
    else
        invalid();
}

```

```

void identifier_list()
{
    if (strcmp(tokarr[cur].type, "id") == 0)
    {
        cur++;
        l1();
    }
    else
        invalid();
}

```

```

void l1()
{
    if (strcmp(tokarr[cur].name, ",") == 0)
    {
        cur++;
        identifier_list();
    }
    else if (strcmp(tokarr[cur].name, "[") == 0)
    {
        cur++;
        if (strcmp(tokarr[cur].type, "num") == 0)
        {
            cur++;

```

```

        if (strcmp(tokarr[cur].name, "}") == 0)
        {
            cur++;
            if (strcmp(tokarr[cur].name, ";") == 0)
            {
                cur++;
                identifier_list();
            }
            else
            {
                return;
            }
        }
        else
            invalid();
    }
}
else
    return;
}

```

```

void statement_list()
{
    /* statement_list -> statement statement_list | e
    FIRST(statement) = { id, if, while } */
    if (strcmp(tokarr[cur].type, "id") == 0
        || strcmp(tokarr[cur].name, "if") == 0
        || strcmp(tokarr[cur].name, "while") == 0)
    {
        statement();
        statement_list();
    }
    else
        return;
}

```

```

void statement()
{
    /* statement -> assign-stat ; | decision-stat | looping-stat */
    if (strcmp(tokarr[cur].type, "id") == 0)
    {
        assign_stat();
        if (strcmp(tokarr[cur].name, ";") == 0)
            cur++;
        else
            invalid();
    }
    else if (strcmp(tokarr[cur].name, "if") == 0)
    {
        decision_stat();
    }
    else if (strcmp(tokarr[cur].name, "while") == 0)
    {
        looping_stat();
    }
    else
        invalid();
}

```

```

void assign_stat()
{

```

```

if (strcmp(tokarr[cur].type, "id") == 0)
{
    cur++;
    if (strcmp(tokarr[cur].name, "=") == 0)
    {
        cur++;
        expn();
    }
    else
        invalid();
}
else
    invalid();
}

```

```

/* decision-stat -> if ( expn ) { statement-list } dprime
dprime      -> else { statement-list } | e      */

```

```

void decision_stat()
{
    if (strcmp(tokarr[cur].name, "if") == 0)
    {
        cur++;
        if (strcmp(tokarr[cur].name, "(") == 0)
        {
            cur++;
            expn();
            if (strcmp(tokarr[cur].name, ")") == 0)
            {
                cur++;
                if (strcmp(tokarr[cur].name, "{") == 0)
                {
                    cur++;
                    statement_list();
                    if (strcmp(tokarr[cur].name, "}") == 0)
                    {
                        cur++;
                        dprime();
                        return;
                    }
                    else
                        invalid();
                }
                else
                    invalid();
            }
            else
                invalid();
        }
        else
            invalid();
    }
    else
        invalid();
}
else
    invalid();
}

```

```

void dprime()
{
    /* dprime -> else { statement-list } | e */
    if (strcmp(tokarr[cur].name, "else") == 0)
    {
        cur++;
    }
}

```

```

    if (strcmp(tokarr[cur].name, "{") == 0)
    {
        cur++;
        statement_list();
        if (strcmp(tokarr[cur].name, "}") == 0)
        {
            cur++;
            return;
        }
        else
            invalid();
    }
    else
        invalid();
}
else
    return; /* e production */
}

/* looping-stat -> while ( expn ) { statement-list } */
void looping_stat()
{
    if (strcmp(tokarr[cur].name, "while") == 0)
    {
        cur++;
        if (strcmp(tokarr[cur].name, "(") == 0)
        {
            cur++;
            expn();
            if (strcmp(tokarr[cur].name, ")") == 0)
            {
                cur++;
                if (strcmp(tokarr[cur].name, "{") == 0)
                {
                    cur++;
                    statement_list();
                    if (strcmp(tokarr[cur].name, "}") == 0)
                    {
                        cur++;
                        return;
                    }
                    else
                        invalid();
                }
                else
                    invalid();
            }
            else
                invalid();
        }
        else
            invalid();
    }
    else
        invalid();
}

void expn()
{
    simple_expn();
    eprime();
}

```

```

}

void eprime()
{
    if (strcmp(tokarr[cur].name, "==") == 0 || strcmp(tokarr[cur].name, "!=") == 0
        || strcmp(tokarr[cur].name, "<=") == 0 || strcmp(tokarr[cur].name, ">=") == 0
        || strcmp(tokarr[cur].name, ">") == 0 || strcmp(tokarr[cur].name, "<") == 0)
    {
        relop();
        simple_expn();
    }
    else
        return;
}

void simple_expn()
{
    term();
    seprime();
}

void seprime()
{
    if (strcmp(tokarr[cur].name, "+") == 0 || strcmp(tokarr[cur].name, "-") == 0)
    {
        addop();
        term();
        seprime();
    }
    else
        return;
}

void term()
{
    factor();
    tprime();
}

void tprime()
{
    if (strcmp(tokarr[cur].name, "*") == 0 || strcmp(tokarr[cur].name, "/") == 0
        || strcmp(tokarr[cur].name, "%") == 0)
    {
        mulop();
        factor();
        tprime();
    }
    else
        return;
}

void factor()
{
    if (strcmp(tokarr[cur].type, "id") == 0)
        cur++;
    else if (strcmp(tokarr[cur].type, "num") == 0)
        cur++;
    else
        invalid();
}

```

```

void relop()
{
    if (strcmp(tokarr[cur].name, "==") == 0)    cur++;
    else if (strcmp(tokarr[cur].name, "!=") == 0) cur++;
    else if (strcmp(tokarr[cur].name, "<=") == 0) cur++;
    else if (strcmp(tokarr[cur].name, ">=") == 0) cur++;
    else if (strcmp(tokarr[cur].name, ">") == 0) cur++;
    else if (strcmp(tokarr[cur].name, "<") == 0) cur++;
    else invalid();
}

```

```

void addop()
{
    if (strcmp(tokarr[cur].name, "+") == 0)    cur++;
    else if (strcmp(tokarr[cur].name, "-") == 0) cur++;
    else invalid();
}

```

```

void mulop()
{
    if (strcmp(tokarr[cur].name, "*") == 0)    cur++;
    else if (strcmp(tokarr[cur].name, "/") == 0) cur++;
    else if (strcmp(tokarr[cur].name, "%") == 0) cur++;
    else invalid();
}

```

```

void main()
{
    FILE *fp = fopen("input.c", "r");
    getNextToken(fp);
    fclose(fp);
    Program();
    if (cur == tokidx)
        valid();
    else
        invalid();
}

```

getToken.c

```

#include <stdio.h>
#include <ctype.h>
#include <string.h>

```

```

typedef struct
{
    int row;
    int col;
    char name[100];
    char type[20];
} token;

```

```

token tokarr[10000];
int tokidx = 0;

```

```

int issymbol(char c)
{
    return (strchr(".,:(){}[]", c) != NULL);
}

```

```

int iskeyword(char *buf)

```

```

{
    char *kw[] = {"auto", "break", "case", "char", "const", "continue", "default",
                  "do", "double", "else", "enum", "extern", "float", "for",
                  "goto", "if", "int", "long", "register", "return", "short",
                  "signed", "sizeof", "static", "struct", "switch", "typedef",
                  "union", "unsigned", "void", "volatile", "while"};
    for (int i = 0; i < 32; i++)
    {
        if (strcmp(buf, kw[i]) == 0)
            return 1;
    }
    return 0;
}

```

```

void getNextToken(FILE *fp)
{
    int c;
    char buf[100];
    int i = 0;
    int rowcnt = 1;
    int colcnt = 0;

    while ((c = fgetc(fp)) != EOF)
    {
        colcnt++;
        if (c == '\n')
        {
            rowcnt++;
            colcnt = 0;
            continue;
        }
        if (isspace(c))
        {
            continue;
        }

        int token_row = rowcnt;
        int token_col = colcnt;
        i = 0;
        buf[i] = '\0';

        if (c == '#')
        {
            while ((c = fgetc(fp)) != EOF && c != '\n')
            {
                colcnt++;
            }
            if (c == '\n')
            {
                rowcnt++;
                colcnt = 0;
            }
            continue;
        }

        // Comments
        if (c == '/')
        {
            int next = fgetc(fp);
            colcnt++;
            if (next == '/')

```

```

{
    while ((c = fgetc(fp)) != EOF && c != '\n')
        colcnt++;
    if (c == '\n')
    {
        rowcnt++;
        colcnt = 0;
    }
    continue;
}
else if (next == '*')
{
    int prev = 0;
    while ((c = fgetc(fp)) != EOF)
    {
        colcnt++;
        if (c == '\n')
        {
            rowcnt++;
            colcnt = 0;
        }
        if (prev == '*' && c == '/')
        {
            break;
        }
        prev = c;
    }
    continue;
}
else
{
    ungetc(next, fp);
    colcnt--;
}
}

// Assignment & Relational operators: =, ==, >, >=, <, <=, !=
if (c == '=')
{
    buf[i++] = c;
    int next = fgetc(fp);
    colcnt++;
    if (next == '=')
    {
        buf[i++] = next;
        buf[i] = '\0';
        strcpy(tokarr[tokidx].type, "Relational operator"); // ==
    }
    else
    {
        ungetc(next, fp);
        colcnt--;
        buf[i] = '\0';
        strcpy(tokarr[tokidx].type, "Assignment operator"); // =
    }
    strcpy(tokarr[tokidx].name, buf);
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}
}

```

```

if (c == '>' || c == '<' || c == '!')
{
    buff[i++] = c;
    int next = fgetc(fp);
    colcnt++;
    if (next == '=')
        buff[i++] = next;
    else
    {
        ungetc(next, fp);
        colcnt--;
    }
    buff[i] = '\0';
    strcpy(tokarr[tokidx].name, buf);
    strcpy(tokarr[tokidx].type, "Relational operator"); // <, <=, >, >=
    if (c == '!' && next != '=') // !=
        strcpy(tokarr[tokidx].type, "Logical operator");
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}

```

```

// Logical operators: &&, ||
if (c == '&' || c == '|')
{
    buff[i++] = c;
    int next = fgetc(fp);
    colcnt++;
    if (next == c)
    {
        buff[i++] = next;
    }
    else
    {
        ungetc(next, fp);
        colcnt--;
    }
    buff[i] = '\0';
    strcpy(tokarr[tokidx].name, buf);
    strcpy(tokarr[tokidx].type, "Logical operator");
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}

```

```

// Arithmetic operators: +, -, *, /, ++, --, +=, -=
if (c == '+' || c == '-' || c == '*' || c == '/')
{
    buff[i++] = c;
    int next = fgetc(fp);
    colcnt++;
    if (next == c && (c == '+' || c == '-'))
        buff[i++] = next;
    else if (next == '=')
        buff[i++] = next;
    else
    {
        ungetc(next, fp);
        colcnt--;
    }
    buff[i] = '\0';
}

```

```

    strcpy(tokarr[tokidx].name, buf);
    strcpy(tokarr[tokidx].type, "Arithmetic operator");
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}

```

```

// Special symbols
if (issymbol(c))
{
    buf[i++] = c;
    buf[i] = '\0';
    strcpy(tokarr[tokidx].name, buf);
    strcpy(tokarr[tokidx].type, "Special symbol");
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}

```

```

// Numbers
if (isdigit(c))
{
    buf[i++] = c;
    while (1)
    {
        int next = fgetc(fp);
        if (next == EOF) break;
        colcnt++;
        if (isdigit(next))
        {
            buf[i++] = (char)next;
        }
        else
        {
            ungetc(next, fp);
            colcnt--;
            break;
        }
    }
    buf[i] = '\0';
    strcpy(tokarr[tokidx].name, buf);
    strcpy(tokarr[tokidx].type, "num");
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}

```

```

// Identifiers & Keywords
if (isalpha(c) || c == '_')
{
    buf[i++] = c;
    while (1)
    {
        int next = fgetc(fp);
        if (next == EOF) break;
        colcnt++;
        if (isalnum(next) || next == '_')
        {
            buf[i++] = (char)next;
        }
        else

```

```

        {
            ungetc(next, fp);
            colcnt--;
            break;
        }
    }
    buf[i] = '\0';
    if (iskeyword(buf))
    {
        strcpy(tokarr[tokidx].name, buf);
        strcpy(tokarr[tokidx].type, "Keyword");
    }
    else
    {
        strcpy(tokarr[tokidx].name, buf);
        strcpy(tokarr[tokidx].type, "id");
    }
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}

if (c == "")
{
    buf[i++] = c;
    while ((c = fgetc(fp)) != EOF)
    {
        colcnt++;
        if (c == '\\')
        {
            buf[i++] = '\\';
            c = fgetc(fp);
            colcnt++;
            if (c != EOF)
                buf[i++] = c;
        }
        else if (c == "")
        {
            buf[i++] = c;
            break;
        }
        else
        {
            buf[i++] = c;
        }
    }
    buf[i] = '\0';
    strcpy(tokarr[tokidx].name, buf);
    strcpy(tokarr[tokidx].type, "Literal");
    tokarr[tokidx].row = token_row;
    tokarr[tokidx++].col = token_col;
    continue;
}
}
}

```

```

int mainhelp()
{
    FILE *fp = fopen("input.c", "r");
    getNextToken(fp);
    fclose(fp);
}

```

```

printf(" %-25s | %-20s | %-4s | %-4s\n", "Name", "Type", "Row", "Col");
printf("-----\n");
for (int i = 0; i < tokidx; i++)
{
    printf(" %-25s | %-20s | %-4d | %-4d\n",
        tokarr[i].name, tokarr[i].type, tokarr[i].row, tokarr[i].col);
}
return 0;
}

```

Input.c

```

#include<stdio.h>
int main()
{
    int a, b, c;
    int x[10], y[5], z;
    a = b+5-c-a*b;
    c = 6*a;
}

```

OUTPUT:

WITHOUT ERROR:

```
CD_A1@CL3-33:~/Documents/230905408$ gcc q1.c -o q1
CD_A1@CL3-33:~/Documents/230905408$ ./q1
Parsing done successfully
CD_A1@CL3-33:~/Documents/230905408$
```

WITH ERROR:

Input.c (with missing semicolon):

```
#include<stdio.h>
int main()
{
    int a, b, c;
    int x[10], y[5], z //missing[];
    a = b+5-c-a*b;
    c = 6*a;
}
```

```
CD_A1@CL3-33:~/Documents/230905408$ ./q1
Invalid, a
r = 6, c = 5
```